

One Person Lab

One Person Lab 白皮书

让复杂知识工作从一次问答走向持续交付

OPL 的核心承诺，是把论文、基金、汇报、专利等复杂成果从一次问答推进到持续审阅、修订和交付。

OPL Framework / One Person Lab App / Foundry Agents

2026-06-08

Public whitepaper

目录

1	定位摘要	2
2	为什么要处理这件事	2
3	OPL 的答案：让 AI 像专业团队一样推进工作	2
4	这套设计从哪里来	3
5	认知计算：阶段内开放式判断	3
6	OPL 的三层产品模型	3
6.1	OPL Framework	3
6.2	One Person Lab App	4
6.3	Foundry Agents	4
7	设计哲学：让 AI 做专家工作，让系统守住边界	4
7.1	AI-first	4
7.2	Executor-first	4
7.3	Stage-led	4
7.4	Contract-light	5
7.5	Owner boundary	5
7.6	Progress-first	5
8	当前十个品牌模块：从上到下的设计骨架	5
8.1	OPL Charter	5
8.2	OPL Atlas	5
8.3	OPL Workspace	6
8.4	OPL Pack	6
8.5	OPL Stagecraft	6
8.6	OPL Runway	6
8.7	OPL Vault	6
8.8	OPL Console	6
8.9	OPL Foundry Lab	7
8.10	OPL Connect	7
9	Foundry Agents：同一骨架，不同领域	7
9.1	MedAutoScience	7
9.2	MedAutoGrant	7
9.3	RedCube AI	7
9.4	OPL Meta Agent	8
9.5	未来 Foundries	8
10	为什么 OPL 能稳定运行	8
11	用户会如何感知 OPL	8
12	这份白皮书如何维护	9
13	参考与编制来源	9

发布日期：2026-06-08

适用对象：希望理解 OPL 品牌、产品定位、设计逻辑和 Foundry Agents 运行方式的用户、合作者、早期采用者和技术决策者。

核心判断：OPL 的核心承诺，是把论文、基金、汇报、专利等复杂成果从一次问答推进到持续审阅、修订和交付。

1 定位摘要

- OPL 面向复杂知识工作的持续交付，让论文、基金、汇报、专利、答辩、审稿等工作以阶段方式推进到成果交付。
- OPL 是正式知识工作的智能体运行框架，围绕阶段、工作空间、证据、权限、恢复和交接设计。
- One Person Lab App 是用户工作台，负责把 Framework 和 Foundry Agents 转化为可见、可操作、可检查的产品体验。
- Foundry Agents 是运行在 OPL 上的专业智能体，例如 Research、Grant、Presentation 和未来的 IP、Award、Thesis、Review。
- OPL 的设计从用户要交付的成果反推到系统合同：用户要的是可靠推进，系统就必须提供阶段、权限、证据、文件、恢复和交接。

2 为什么要处理这件事

AI 已经很擅长回答一个问题、生成一段代码或润色一份材料。当任务进入论文、基金、汇报、专利、答辩和审稿这类长周期成果时，用户关注的是整项工作如何持续推进到交付。

一个人做研究、写基金、准备汇报、整理专利或推进答辩材料时，材料很多，工作跨越很多天甚至很多周，中间会反复修改、审阅、补证据、换方向、重开上下文。多轮推进之后，用户需要清楚知道当前阶段、最新版文件、已审阅判断和待处理阻塞。

项目管理工具擅长记录任务名和状态；固定自动化擅长执行确定步骤；AI 对话擅长生成内容。长周期知识工作还需要一套能承载阶段、证据、文件、负责人和交付边界的运行结构，让理解、比较、创作、审阅和修订可以在同一工作线上持续发生。

OPL 为高价值知识工作提供这样的操作系统：一个人也可以拥有类似专业团队的持续推进能力，AI 围绕阶段目标组织材料、工具和候选方案，系统把复杂成果一步步推到可以检查、可以修改、可以交付的位置。

- 多轮推进之后，当前阶段如何持续记录？
- 材料、文件、修改和证据如何形成可回溯链路？
- 准备、执行、审核、修订和交付如何保持清楚边界？
- 用户离开电脑后，任务如何保留运行线索，并在回来时呈现进展、阻塞和下一步？
- 多个专业 Agent 如何共享一套运行、文件、进度和交付体系？

3 OPL 的答案：让 AI 像专业团队一样推进工作

One Person Lab 正是围绕这些问题设计的。它把复杂知识工作拆成一个个能推进的阶段：准备材料、执行创作、质量审核、修订完善、交付收口。

每个阶段都围绕一个真实成果增量工作。AI 可以在同一阶段里整理资料、提出候选方案、比较取舍、调用工具、接受审阅并继续修订。用户通过工作台看到进度、文件、证据、阻塞和下一步都被清楚保留下来。

对话工具擅长处理当次问题；OPL 把关注点推进到整项工作的阶段、产物、回执、负责人和工作空间。用户对工作的信任来自可检查的阶段记录和交付物。

在这个模型里，AI 保持专家判断空间，可以理解材料、比较方案、创造内容、接受反馈、继续修订。系统负责让这些开放式工作具备边界、记录和可恢复落点。

- 用户看到任务进展、当前文件和交付物。
- Agent 接收阶段目标、材料上下文和可用能力。
- 系统保存证据、回执、阻塞和恢复点。

4 这套设计从哪里来

OPL 的当前品牌模块来自长周期知识工作的真实摩擦：一次性回答需要延伸为持续交付，自动化脚本需要连接专家判断，界面可见性需要连接质量完成。

第一层来源，是 OPL 自己要服务的任务：论文、基金、汇报、专利、报奖、学位论文、审稿和修回。这些任务都有明确交付物，也都有大量中间判断。它们要求 AI 能做专业工作，同时要求系统能说清楚当前阶段、材料来源、文件位置、质量门、阻塞原因和下一步。

第二层来源，是 MAS、MAG、RCA 等 Foundry Agents 的真实实践。医学研究需要证据包、稿件、发表质量门；基金写作需要方向判断、申请书结构、模拟评审和修订；视觉交付需要故事线、渲染、审阅和导出。不同领域的内容不同，但它们都需要同一类运行外围：阶段、工作空间、回执、阻塞、产物 lineage、状态投影和恢复机制。

第三层来源，是成熟工程系统的分层经验。OPL 借鉴 catalog、durable execution、operator pattern、artifact lineage、machine-readable descriptor 和 ADR 这样的治理思想，并把这些原则落到 One Person Lab 自己的用户路径、专业智能体和交付边界里。

- 从用户痛点来：复杂成果从一次回答延伸为持续推进。
- 从领域实践来：MAS、MAG、RCA 证明不同专业 agent 需要共享运行骨架。
- 从工程经验来：可发现、可恢复、可审计、可派生的系统才容易长期维护。

5 认知计算：阶段内开放式判断

固定自动化流程擅长处理确定步骤：先做 A，再做 B，最后输出 C。复杂知识工作需要更强的阶段内判断：写一篇文章、改一个基金本子、做一套正式汇报，过程中经常需要反复判断、比较、推翻、重写和审阅。

OPL 把这种阶段内的开放式专家工作称为认知计算：AI 在一个可观察、可接力的阶段里完成理解、比较、创作、审阅和修订。系统提供目标、材料、工具边界、权限、质量门和交接要求，让执行者在阶段内自主组织工作。

这样设计的好处是，AI 执行者越强，OPL 的阶段能力就越强。模型升级、提示词改进、技能改进、领域知识改进，都能直接提升阶段内的真实工作能力；同时，OPL 的阶段、回执、工作空间、状态投影和恢复机制让这些能力转化为可追踪的持续工作。

用户最终看到的是工作在往前走：下一版文件形成了，证据能解释，审阅有边界，阻塞能被看见，交接能继续。

- OPL 保留专家工作所需的开放判断空间。
- OPL 让 AI 在阶段内有足够空间完成真实判断。
- OPL 用系统边界保证开放式工作可见、可复核、可恢复。

6 OPL 的三层产品模型

OPL 对外呈现为一个统一品牌，对内保持三层清楚分工：Framework、App、Foundry Agents。这样的结构让用户体验、运行框架和专业智能体可以各自演进，并通过共同合同保持一致。

默认运行链路

```
用户 / App / CLI
  -> Codex CLI 标准执行者
    -> OPL 显式激活
      -> provider-backed 阶段运行
        -> 选定的 Foundry Agent
          -> 领域事实 / 质量门 / 交付物
```

6.1 OPL Framework

运行框架

Framework 是让复杂工作能跑起来并持续跑下去的底座。它负责执行会话、显式激活、阶段控制、任务队列、长跑运行、共享合同、恢复和审计。它让任务可以被启动、恢复、投影和收口。

- 默认第一公民 executor 是 Codex CLI。
- Temporal-backed provider 是生产级在线运行的必需底座。
- Framework 负责通用运行外围，领域事实、质量结论和产物权威由对应 Foundry Agent 持有。

6.2 One Person Lab App

用户工作台

App 是 OPL 的产品入口。它把运行状态、工作空间和 Foundry Agent 投影组织成可操作工作台：选择任务、查看进度、打开文件、处理阻塞、接收更新。它把任务、阶段、阻塞、交付物、来源引用和下一步动作呈现为清楚界面。

- App 固定 Codex CLI 作为标准执行路径。
- App 负责界面产品事实、发布门、用户教程和可用性验证。
- App 负责展示和交互，质量完成结论回到对应领域智能体。

6.3 Foundry Agents

领域智能体

Foundry Agents 是 MAS、MAG、RCA、OMA 以及未来 Patent、Award、Thesis、Review 等领域智能体。它们像专业团队里的不同角色：研究、基金、视觉交付、智能体构建各自有材料理解、审阅标准和交付边界。它们以统一 OPL 结构运行，并保留自己的领域判断、质量门、产物权威和直接使用路径。

- 同一生命周期：材料进入、阶段执行、质量门、负责人回执、交付物交接。
- 不同领域保留不同输入、输出、知识、评价标准和权威动作。
- OPL 通过引用投影领域状态，领域正文和产物内容由对应智能体管理。

7 设计哲学：让 AI 做专家工作，让系统守住边界

OPL 最重要的设计逻辑，是同时保留 AI 的自由度和系统的纪律。AI 在阶段内完成开放式专家工作，系统提供可审计、可恢复、可交接的边界。

7.1 AI-first

能力来自更强执行者

复杂知识工作交给 AI 执行者做开放式判断、写作、评审和修订。代码提供目标、上下文、权限、工具和质量边界，把专家行为留给最合适的执行者。这就是 OPL 能跟随更强模型、更好提示词、更好技能一起变强的原因。

- 模型升级、提示词改进、技能改进会直接提升系统能力。
- 合同定义安全和可恢复下限，专家判断保留在阶段执行与质量门中。

7.2 Executor-first

阶段内最小执行单位是 Agent executor

OPL 让一个被选中的执行者在一个阶段内完成接近人类专家的真实工作包。阶段把目标、上下文、工具和质量边界聚合在一起，让成果推进保持连续。

- 默认执行者是 Codex CLI。
- 其他执行者通过显式 adapter 接入，并必须提供回执、审计和失败保护语义。

7.3 Stage-led

大型任务按阶段推进

论文、基金、视觉交付和答辩材料都有自然阶段。OPL 把阶段作为可观察、可恢复、可审计的工作单元。阶段让 AI 有空间完成专家工作，也让用户能看到工作推进到了哪里。

- 每个阶段有目标、输入、输出、知识、工具边界和质量门。
- 阶段收口回到负责人回执、结构化阻塞、人工确认或路线调整。

7.4 Contract-light

合同只守安全和可恢复下限

合同负责责任方、权限、安全动作、回执、阻塞、审计、恢复和状态投影。质量结论由独立审阅者、领域质量门或负责人回执给出；就绪状态聚合启动条件和证据缺口。这样做让机械检查、运行状态和专家质量判断各有边界。

- 机械检查只能定位缺口。
- 质量裁决来自独立审阅者、领域质量门或负责人回执。

7.5 Owner boundary

每个判断都有责任方

Framework、App 和 Foundry Agents 各有自己的权威边界。OPL 管通用运行外围，领域智能体管领域事实，App 管用户界面和发布证据。清楚的责任边界是长期可维护的前提。

- 每类事实都有单一责任方。
- 界面和运行框架消费领域投影，领域质量由对应智能体裁决。

7.6 Progress-first

用户需要看见下一步

OPL 默认把责任方变化放在最前面：当前等待谁、需要什么交付增量、缺什么回执或阻塞说明。用户首先看到可推进状态，然后再按需要展开事件、trace 和诊断信息。

- 进度来自产物、证据、决策和交接。
- 诊断细节在展开详情中呈现。

8 当前十个品牌模块：从上到下的设计骨架

当前十个模块是 OPL Framework 的能力分区。它们把顶层理念落实到真实运行：先建立语言和边界，再组织目录和工作空间，再定义 domain pack 和 authority ABI，再设计阶段、长跑执行、证据回执、用户控制台、智能体工坊和外部连接。

8.1 OPL Charter

语言和边界

Charter 固定 OPL 的顶层定位、命名、产品层级、ADR/RFC 和品牌组合治理。它回答什么叫 OPL、为什么存在、什么属于 Framework、什么属于 App、什么属于 Foundry Agent。Charter 让长期演进始终沿着同一套语言展开。

- 让整个系统拥有统一语言。
- 让路线演进、命名和品牌表达保持一致。

8.2 OPL Atlas

可发现目录

Atlas 管理智能体、能力、入口、责任方、依赖和生命周期目录。用户需要知道 OPL 能做什么，系统需要知道能力来自哪里、谁负责、如何调用、当前生命周期是什么。Atlas 把这些信息放进可发现目录。

- 负责能力发现和目录组织。
- 把领域描述、模块注册和公开入口放进统一目录。

8.3 OPL Workspace

用户和 Agent 共同检查的项目空间

Workspace 把材料、共享资源、阶段输出、交接和文件生命周期组织成可检查结构。复杂任务最终必须落到文件、材料和交付物上；用户能在项目空间里看到当前产物和交付物。

- 默认模型是 Workspace Group -> Project Unit -> Stage Artifact Unit。
- 真实完成绑定清单、回执和当前指针。

8.4 OPL Pack

Domain Pack 与 authority ABI

Pack 定义 Foundry Agent 如何声明自己的 domain pack、stage/action refs、authority functions、generated surface 输入和 pack compiler 对齐状态。它回答一个领域智能体怎样被 OPL 标准化接入，同时不让 OPL 接管领域 handler、owner receipt 或质量裁决。

- 固定 domain pack、authority ABI 和 generated surface 输入。
- 让标准智能体更容易生成、验证、分发和维护。

8.5 OPL Stagecraft

阶段内认知计算设计

Stagecraft 负责阶段设计、提示词、技能、工具可用性、知识、评价标准和独立质量门。它回答这个阶段要让 AI 怎样像专家一样工作，并在保留开放式专家空间的同时，让输入、输出和边界可审计。

- 工具目录说明可用能力、权限和边界。
- 复杂质量判断进入独立阶段或质量门。

8.6 OPL Runway

可恢复的长跑执行

Runway 负责持久执行、类型化队列、尝试记录、租约、重试/死信、唤醒和人工确认。它回答这项工作怎么持续跑下去。长期任务通过 Runway 保留运行线索、恢复点和人工介入边界。

- 承接 provider-backed 阶段运行。
- 承载长跑执行和恢复语义，领域结论回到对应 agent。

8.7 OPL Vault

证据、回执和 lineage

Vault 保存证据引用、回执引用、结构化阻塞引用、产物 lineage、恢复证明和来源记录。它回答为什么我们相信当前状态。任务为什么可以继续、为什么被阻塞、如何恢复，都有可追踪依据。

- 保存引用和 lineage，让证据可追踪。
- 领域正文、记忆和产物内容保留在对应智能体的权威边界内。

8.8 OPL Console

用户和 operator 的可见控制台

Console 把就绪状态、当前责任方、下一步动作、阻塞、产物和展开详情投影到 App/operator 工作台。它回答用户现在应该看什么、做什么，并把运行状态转化为可行动信息。

- 默认优先呈现责任方变化。
- 消费投影视图，并把运行状态呈现为工作台视图。

8.9 OPL Foundry Lab

智能体创建与机制改进

Foundry Lab 支撑新智能体创建、测试接管、机制改进、canary、promotion 和 rollback。它回答 OPL 怎样持续变强。运行证据可以转成可执行改进任务，帮助智能体机制持续演进。

- 用于改进智能体机制。
- 与 MAS、MAG、RCA 或未来智能体的领域权威保持清楚分工。

8.10 OPL Connect

外部调用和分发连接

Connect 把同一合同派生到 CLI、MCP、OpenAI/AI SDK tools、Skill/plugin、模块安装、发布/安装分发和漂移检查。它回答同一套能力怎样被安装、发现、调用和分享。

- 让能力可以被安装、同步、发现和调用。
- 保持外部调用与领域自有语义一致。

9 Foundry Agents: 同一骨架，不同领域

Foundry Agents 让 OPL 的品牌进入具体专业场景：研究、基金、视觉交付、智能体构建，以及未来的专利、报奖、论文和审稿。每个 agent 都面向一个高价值 workflow，拥有自己的领域知识、质量门和交付权威，同时共享 OPL 的 stage-led 运行骨架。

9.1 MedAutoScience

Research Foundry

面向医学和科研论文 workflow，从数据、文献、研究问题、分析、证据包到稿件和投稿包。研究工作特别需要证据、审阅、修订和投稿边界，所以 MAS 持有研究事实、发表质量门、产物权威和负责人回执。

- 用户感知：把研究项目持续推进到可审阅论文和投稿材料。
- OPL 承载阶段尝试、队列、回执投影和工作空间/运行支撑。

9.2 MedAutoGrant

Grant Foundry

面向基金方向判断、申请书写作、作者侧模拟评审和修订。基金工作需要重新组织问题、创新点、技术路线和评审说服力；MAG 复用研究证据和记忆，同时保持基金事实与可资助性评审边界。

- 用户感知：从想法到申请书草稿、模拟评审包和修订计划。
- 质量判断回到 MAG 自己的质量门和负责人回执。

9.3 RedCube AI

Presentation Foundry

面向视觉交付、PPT、报告、storyboard、render、review 和 export。汇报材料需要叙事、结构、视觉和可展示文件同时成立；RCA 持有视觉事实、审阅/导出结论和产物权威。

- 用户感知：把材料转成可展示的视觉交付物。
- OPL 负责阶段文件夹、产物清单、回执引用和投影。

9.4 OPL Meta Agent

Foundry Lab managed module

面向智能体创建、测试接管、机制改进和目标智能体工单。它帮助 OPL family 更系统地生产智能体、测试智能体，并从失败中改进机制；领域智能体的最终事实仍由对应领域责任方持有。

- 用户感知：OPL 能持续变强、持续生成和修复自己的 agents。
- 边界：生成改进工单或结构化阻塞，领域责任方负责最终签收。

9.5 未来 Foundries

IP / Award / Thesis / Review

IP、Award、Thesis、Review 等工作流会沿用同一骨架：材料进入、领域包解释、阶段式执行、独立质量门、负责人回执或结构化阻塞、交付物交接、OPL 投影。它们会在对应领域边界成熟后成为新的 Foundry。

- 每个新智能体都有自己的领域事实和质量边界。
- 共享 OPL runtime，并把领域差异保留在对应 Foundry 内。

10 为什么 OPL 能稳定运行

OPL 的稳定感来自一组可靠边界：开放式智能工作在阶段内发生，系统负责留痕、交接、恢复和质量边界。它让 AI 像专家一样工作，也让系统像专业组织一样运行。

第一，任务以阶段为单位推进。一个阶段足够大，可以让 AI 执行者完成真实工作包；同时也足够清楚，可以定义输入、输出、权限、质量门和收口形态。

第二，责任边界清楚。Framework 持有框架事实；App 持有产品和展示事实；Foundry Agent 持有领域事实、质量结论和产物权威。每个判断都有归属，投影、运行和质量结论保持清楚分工。

第三，证据以引用和 lineage 方式保存。OPL 记录回执、结构化阻塞、lineage、当前指针和恢复证明，让任务可以被审计和恢复；领域正文、产物内容和质量裁决保留在对应责任方处。

第四，用户默认看到下一步动作。工作台优先回答当前等待谁、缺什么、下一步能做什么，再提供事件、trace、计数和诊断详情。

第五，系统从单一来源派生多个入口。CLI、App、Skill、MCP、AI SDK tool、plugin 和 release/install 分发都围绕同一 contract surface 生成，避免每个入口各说各话。

第六，设计始终从用户要交付的成果反推。默认入口聚焦进度、产物、证据、阻塞、责任方和下一步；诊断细节进入展开详情和维护面。这样的减法治理让 OPL 保持清楚、专业和可维护。

- 这就是 OPL 的核心体验：AI 有足够自由度完成专家工作，系统有足够纪律保证工作可见、可复核、可恢复、可交付。
- 用户对工作的信任来自阶段、产物、回执、责任方和工作空间。

11 用户会如何感知 OPL

OPL 的用户体验目标，是让复杂知识工作呈现为一条可以持续推进、可以检查、可以交接的工作线。

在 App 中，用户选择任务和 Foundry Agent。标准路径使用 Codex CLI 执行者；运行和 provider 状态被组织成就绪状态、当前责任方、下一步动作和展开详情。

在工作空间中，用户看到材料、阶段输出、交付物和当前指针。运行状态解释工作如何推进，项目文件承载可检查、可复制、可分享和可归档的交付物。

在长期任务中，用户看到每个阶段的进度、阻塞和交接。任务可能需要人类批准、补材料、接受路线调整或等待领域负责人回执，但这些状态都会显式出现。

用户从任务入口进入，查看当前阶段，打开产物，处理必要阻塞，并在关键节点做判断。复杂运行细节由系统组织，面向用户呈现为清楚的进度、文件、证据和下一步。

- OPL 把复杂性收进系统，把可行动信息呈现给用户。
- OPL 把 AI 的创造力留在阶段内，把责任和证据留在系统边界上。
- OPL 把一次性助手升级成可以持续运行的实验室伙伴。

12 这份白皮书如何维护

白皮书本身也遵循 OPL 的维护哲学：单一内容源、可复现生成、生成后验证。

内容源是 docs/public/whitepaper/opl-whitepaper.source.json。Markdown、PDF 和 verification JSON 都从这一份 JSON 派生。这样做可以避免同一份白皮书在网页、PDF 和仓库文本中出现多套不一致的正文。

PDF 使用 Pandoc + XeLaTeX 生成，字体采用本机可用的 Noto Sans CJK SC。生成脚本会渲染 PDF 页面、抽取文本、检查关键内容、记录页数和工具链路径，方便后续维护者确认产物仍可分享。

外部白皮书风格参考了经典项目的做法：白皮书先建立定位和信念，再解释系统模型；同时保留当前性边界，把历史、愿景和已经验证的 production-ready 声明分开表达。

OPL 自己的 README 是这份白皮书的主要语气来源：先提出用户为什么需要 One Person Lab，再解释阶段式推进、专业 agent、进度证据和产品分层。白皮书比 README 更完整地交代设计来源和品牌模块结构，但重心仍然是让用户明白 OPL 为什么存在、为什么必须这样设计。

- 正文更新从内容源开始。
- 生成脚本统一派生 Markdown、PDF 和验证记录。
- 分享版通过 PDF 渲染页和 verification JSON 检查。

13 参考与编制来源

- [Ethereum Whitepaper](#)：参考其白皮书页面对愿景、系统模型和当前性说明的组织方式。
- [Bitcoin whitepaper PDF](#)：参考其短而正式的技术宣言式结构。
- [Pandoc User Guide](#)：本仓采用 Pandoc + XeLaTeX 生成 PDF。
- [Typst PDF documentation](#)：作为后续可能迁移到更现代 PDF 工具链的参考。